

■ Lab 1 — Préparation de l'Espace de Travail (GitPod)

■ Objectifs du Lab

À la fin de ce lab, chaque participant aura :

- Un compte **GitPod** opérationnel
 - Un **environnement de développement cloud** avec Terraform et AWS CLI pré-installés
 - Ses **credentials AWS** configurés et vérifiés
 - Un environnement prêt pour tous les Labs suivants
-

■ Étape 0 — Prérequis

Avant de commencer, assurez-vous d'avoir :

- Un **navigateur moderne** (Chrome, Firefox, Edge)
- Un compte **Google** ou **GitHub** pour vous connecter à GitPod
- Les **informations d'accès AWS** fournies par le formateur :
 - `AWS_ACCESS_KEY_ID`
 - `AWS_SECRET_ACCESS_KEY`
 - `AWS_DEFAULT_REGION` (ex : `eu-west-1`)

■ ■ *GitPod est un IDE 100% cloud dans votre navigateur — aucune installation nécessaire sur votre poste.*

■ Étape 1 — Créer un compte GitPod

1. Ouvrez ■ <https://gitpod.io>
2. Cliquez sur "**Continue with Google**" ou "**Continue with GitHub**"
3. Autorisez GitPod à accéder à votre compte
4. Votre compte GitPod est créé automatiquement

■ Le plan ****gratuit GitPod**** inclut 50 heures/mois — largement suffisant pour la formation.

■ Étape 2 — Ouvrir l'environnement de formation

GitPod peut ouvrir directement n'importe quel repo GitHub en un clic.

1. Ouvrez le lien suivant dans votre navigateur :

■ <https://gitpod.io/#https://github.com/SAIDI-HAMZA/tf-training-template>

2. GitPod démarre automatiquement un environnement basé sur le repo de formation

3. Attendez **2 à 3 minutes** que l'environnement se construise (première fois uniquement)

■ Notre repo contient déjà un fichier `.devcontainer/devcontainer.json` — GitPod le détecte automatiquement et configure l'environnement avec Terraform et AWS CLI pré-installés.

■ Si GitPod vous demande de choisir un éditeur, sélectionnez **VS Code Browser** pour rester 100% dans le navigateur.

■ Étape 3 — Vérifier les outils installés

Une fois l'environnement démarré, ouvrez le terminal intégré :

- Menu **Terminal** → **New Terminal**, ou raccourci `ctrl+```

Exécutez les commandes suivantes :

```
# Vérifier Terraform
terraform version
```

Résultat attendu :

```
Terraform v1.15.5
on linux_amd64
```

```
# Vérifier AWS CLI
aws --version
```

Résultat attendu :

```
aws-cli/2.x.x Python/3.x.x Linux/...
```

```
# Vérifier jq
jq --version
```

■ Si une commande n'est pas reconnue, signalez-le au formateur.

■ Étape 4 — Configurer l'accès AWS

Méthode A — Variables d'environnement (recommandée)

Dans le terminal GitPod, exportez vos credentials :

```
export AWS_ACCESS_KEY_ID="AKIAXXXXXXXXXXXXXXXXXXXX"
export AWS_SECRET_ACCESS_KEY="XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"
export AWS_DEFAULT_REGION="eu-west-1"
```

Pour les rendre **persistantes** dans votre session GitPod :

```
echo 'export AWS_ACCESS_KEY_ID="AKIAXXXXXXXXXXXXXXXXXXXX"' >> ~/.bashrc
echo 'export AWS_SECRET_ACCESS_KEY="XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"' >> ~/.bashrc
echo 'export AWS_DEFAULT_REGION="eu-west-1"' >> ~/.bashrc
source ~/.bashrc
```

Méthode B — Variables d'environnement GitPod (persistantes entre sessions)

GitPod permet de stocker des variables d'environnement de façon sécurisée au niveau du compte :

```
# Stocker les credentials de façon permanente dans GitPod
gp env AWS_ACCESS_KEY_ID="AKIAXXXXXXXXXXXXXXXXXXXX"
gp env AWS_SECRET_ACCESS_KEY="XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"
gp env AWS_DEFAULT_REGION="eu-west-1"
```

■ Avec `gp env`, les credentials sont **automatiquement disponibles** dans tous vos futurs environnements GitPod, sans avoir à les re-saisir.

■ Étape 5 — Vérifier la connexion AWS

```
aws sts get-caller-identity
```

Résultat attendu :

```
{
  "UserId": "AIDAXXXXXXXXXXXXXXXXXX",
  "Account": "123456789012",
  "Arn": "arn:aws:iam::123456789012:user/terraform-training-alice"
}
```

```
# Vérifier les droits EC2
aws ec2 describe-regions --output table
```

■ Si vous obtenez un JSON avec `Account` et `Arn`, votre accès AWS est opérationnel.

■ Si vous obtenez une erreur `InvalidClientTokenId`, vérifiez vos credentials avec le formateur.

■ Étape 6 — Reprendre un environnement existant

GitPod arrête automatiquement votre environnement après 30 minutes d'inactivité. Pour le reprendre :

1. Allez sur ■ <https://gitpod.io/workspaces>
2. Retrouvez votre environnement dans la liste
3. Cliquez sur "Open" pour le relancer

■ Vos fichiers sont ****sauvegardés automatiquement**** entre les sessions — vous reprenez exactement là où vous en étiez.

■ Validation du Lab

#	Critère	Validé
1	Compte GitPod créé (Google ou GitHub)	■
2	Environnement ouvert via le lien GitPod	■
3	`terraform version` retourne `v1.15.5`	■
4	`aws --version` retourne une version v2.x	■
5	`aws sts get-caller-identity` retourne votre ARN	■
6	`gp env` configuré pour persister les credentials	■

■■ Résolution des problèmes courants

Problème	Cause probable	Solution
Environnement bloqué au démarrage	Première construction longue	Patienter 3–5 min, actualiser la page
`terraform: command not found`	Conteneur non terminé	Fermer/rouvrir le terminal
`InvalidClientTokenId`	Mauvais Access Key ID	Vérifier la valeur (pas d'espace)
`AuthFailure`	Mauvais Secret Access Key	Re-copier le secret depuis le formateur
Environnement introuvable	Session expirée	Aller sur gitpod.io/workspaces et rouvrir
Éditeur ne s'ouvre pas	Bloqueur pub ou extension navigateur	Désactiver les extensions et réessayer

■ ****Fin du Lab 1 — Votre environnement GitPod est prêt !****

Le Lab 2 commence ici : vous allez écrire votre premier fichier Terraform avec le bloc `provider` et déployer une ressource AWS.